Counselor NIDS

How Each Methodology Step Was Done, and How the Code Does It

For a colleague reading the project: each stage explains what we did and the calculation, the file and code that carries it out line by line, and a worked example with real numbers. Every stage is tagged with the objective and research question it serves, so this document also shows how the methodology answers the problem statement, the objectives and the research questions.

What the project solves (referenced by the tag on every stage below)

Problem. Existing systems combine rules and machine learning too simply and can teach themselves to ignore new attacks, so **previously unseen attacks are missed** while false positives stay high and alerts are unexplained.

Objectives. O1 design the architecture · O2 implement it on corrected CIC-IDS2017/2018 with one flow extractor · O3 evaluate unseen-attack detection (leave-one-family-out, cross-dataset) · O4 measure verification's effect on false positives · O5 evaluate safeguarded adaptation on a new network without forgetting · O6 assess explanation quality.

Research questions. RQ1 detect unseen attacks vs baselines · RQ2 does independent verification cut false positives without losing unseen attacks · RQ3 do the novelty score and "AI-only" find unseen attacks · RQ4 does safeguarded adaptation improve a new network without forgetting · RQ5 are the explanations faithful.

1. Data collection and flow extraction

Addresses → Objective O2 — same flow extractor for training and live operation.

Method. A capture is a raw stream of packets, but a model needs a fixed-size description of each connection, so we reconstruct connections and summarise them. Packets are grouped by their 5-tuple (source/destination address, source/destination port, protocol) into bidirectional *flows*, and each flow is turned into ~80 numbers: flow duration = (last packet time – first packet time); total forward packets = count of packets from source to destination; inter-arrival features summarise the gaps between successive packets; TCP-flag counts tally how many packets carried SYN, ACK, FIN, etc. The **same** extractor is used for the datasets and for live capture.

In code → src/nids/capture/flowmeter.py (Python cicflowmeter) · src/nids/datasets/flow_features.py

```
features = list(CIC2018_TO_2017.values())    # the ~77 shared flow features
# the flow meter groups packets by 5-tuple and emits one row of features per flow
```

The code, step by step:

- `CIC2018_TO_2017` maps the 2018 feature names onto the 2017 names, giving **one shared feature vocabulary** so both datasets describe a flow the same way.
- The flow meter (a patched `cicflowmeter`) does the grouping and the arithmetic; its output is one DataFrame row per connection, with a label column.

Worked example. A connection sends 20 packets between 10:00:00.000 and 10:00:04.000. The extractor records flow duration = 4.000 s, total packets = 20, and (if 12 went one way) fwd packets = 12 — three of its ~77 numbers.

2. Cleaning and standardising

Addresses → Objective O2 · underpins **all RQs** — removes duplicates and prevents leakage.

Method. Rate features (bytes ÷ duration) can be infinite when a flow lasts almost no time — those rows are dropped. **Exact-duplicate** rows are removed so one flow cannot sit in both training and test. Then each feature is put on a common scale: a signed logarithm first compresses the range, and a standardiser maps $x \to (x-\mu)/\sigma$. The scaler's μ and σ are learned on **training rows only**.

In code → scripts/live_detectors/data_prep.py : prepare() · src/nids/detector/preprocessing.py

```
flows = flows.replace([np.inf,-np.inf], np.nan).dropna(subset=features)
flows = flows.drop_duplicates(subset=features)          # remove exact duplicates
flows['is_attack'] = flows['label'] != 'Benign'         # binary target
# build_preprocessor(): signed_log1p -> StandardScaler -> VarianceThreshold (fit on train)
```

The code, step by step:

- **Line 1** turns infinities into NaN and drops those rows — the undefined rates.
- **Line 2** compares the whole 77-feature vector, so only genuine repeats are removed (about one flow in six in our data).
- **Line 3** collapses the many attack labels into a single attack/benign target.
- The preprocessor log-compresses, then standardises, then drops constant columns; fitting it inside `DetectorModel.fit` on training keeps the test set's statistics out of training.

Worked example. A flow with 5,000 bytes: signed-log gives ln(1+5000) ≈ 8.52. If, across training flows, that logged feature has mean 6.0 and std 1.2, the standardised value is (8.52–6.0)/1.2 ≈ **+2.1** — this flow is 2.1 standard deviations above average on that feature.

3. Three-way, time-ordered split (60 / 20 / 20)

Addresses → Underpins **all RQs** — the leakage-free split that answers the problem statement's "weak evidence" gap.

Method. Within each (attack type, capture day) we sort flows by start time and cut at 60% and 80%: the earliest 60% is training, the next 20% validation, the last 20% test. Splitting **by time** means the model never trains on traffic recorded after what it is tested on; splitting **within each label** means every attack family appears in all three sets. The test set is used once.

In code → scripts/live_detectors/data_prep.py : three_way_split()

```
flows = flows.sort_values('flow_start', kind='stable')
for _, g in flows.groupby(['label','capture']):
    a, b = int(len(g)*0.6), int(len(g)*0.8)
    train.append(g.iloc[:a]); val.append(g.iloc[a:b]); test.append(g.iloc[b:])
```

The code, step by step:

- The **stable** sort keeps flows with the same timestamp in a fixed order, so the split is reproducible.
- Grouping by `(label, capture)` and cutting each group guarantees the 60/20/20 proportions **per family**, not just overall.
- `iloc[:a] / iloc[a:b] / iloc[b:]` take the earliest, middle and latest slices.

Worked example. A day has 10,000 DDoS flows. a = 6,000 and b = 8,000, so the earliest 6,000 train the model, the next 2,000 tune its thresholds, and the latest 2,000 — which happened *after* the training traffic — are the untouched test.

4. K-Means clustering (inside each detector)

Addresses → Objective O1 — the specialist-detector design.

Method. Each detector partitions its traffic into clusters so different regions can be judged by different classifiers. K-Means places K centres, assigns each flow to its nearest centre by Euclidean distance ($\sqrt{\Sigma(xi-ci)^2}$), moves each centre to the mean of its flows, and repeats until stable.

In code → src/nids/detector/model.py : DetectorModel.fit() / assign_clusters()

```
self.kmeans = KMeans(self.n_clusters, n_init=10, random_state=seed).fit(Xt_eval)
clusters = self.kmeans.predict(Xt)        # nearest centroid for each flow
```

The code, step by step:

- `n_init=10` restarts K-Means ten times and keeps the best, so one unlucky start doesn't decide the clusters.
- `predict(Xt)` assigns any new flow to the nearest centre — the same operation used at detection time and for the novelty score.

Worked example. With two centres c1 and c2 and a flow at distances 3.2 and 9.8, the flow is put in cluster 1 (3.2 is nearer). After all flows are assigned, c1 moves to the average position of the flows assigned to it.

5. Train the classifiers, select the best per cluster

Addresses → Objective O1 — the specialist-detector design.

Method. Five classifiers (Random Forest, three Decision Trees, Naive Bayes) are trained on the training flows. Then, on the **validation** flows, each classifier's accuracy is measured *per cluster* — correct ÷ flows in that cluster — and each cluster keeps the classifier(s) within a small margin α of its best.

In code → src/nids/detector/model.py : fit() + reselect()

```
accuracy = {name: float(np.mean(p[in_cluster] == y_eval[in_cluster]))
            for name, p in predictions.items()}        # accuracy per cluster on VALIDATION
best = max(profile.accuracy.values())
profile.selected = [n for n,a in profile.accuracy.items() if a >= best - alpha]
```

The code, step by step:

- `p[in_cluster] == y_eval[in_cluster]` is a true/false array of whether each validation flow in the cluster was classified correctly; its mean is the accuracy.
- Keeping classifiers within `alpha` of the best lets near-tied classifiers all vote — which is what creates the conflicts that trigger consultation.

Worked example. In one cluster the validation accuracies are RF 0.95, DT1 0.94, DT2 0.90, NB 0.62. With α = 0.02 the best is 0.95 and the keep-line is 0.93, so this cluster votes with **{RF, DT1}** and drops the weaker two.

6. A detector's attack probability and verdict

Addresses → Objective O1 · feeds **RQ1** (the ML verdict).

Method. For a flow we find its cluster and average the kept classifiers' attack probabilities, p = (p1+…+pn) ÷ n; the verdict is the majority vote.

In code → src/nids/counselor/decision.py : detector_proba() / _attack_proba() · src/nids/detector/detector.py : detect()

```
probs = np.column_stack([_attack_proba(m.classifiers[n], Xt[rows]) for n in names])
out[rows] = probs.mean(axis=1)            # mean attack probability = p_d(x)
```

The code, step by step:

- `_attack_proba(clf, X)` is a classifier's probability of the **attack** class for every flow in the cluster.
- When a classifier was trained on **benign flows only** — which happens in the leave-one-family-out runs (§11), where a detector's entire attack family is withheld — the attack class is absent from `predict_proba`, so it contributes **0** rather than raising an error. This keeps the unseen-attack experiment runnable.
- `column_stack(...).mean(axis=1)` averages those probabilities — one number per flow.

Worked example. If the kept classifiers output 0.90, 0.70 and 0.80, the detector's probability is (0.90+0.70+0.80)/3 = **0.80**, and the majority vote (all above 0.5) is **attack**.

7. Novelty score and its threshold

Addresses → Objectives O1, O3 · Research question **RQ3** (and RQ1) — the unseen-attack signal.

Method. The novelty score is how far a flow sits from the nearest thing the detector learned: n(x) = min over centres of the Euclidean distance. We set τ to the 95th percentile of benign validation scores, so only the 5% most-distant benign flows count as novel. A flow with n(x) > τ is novel — a possible unseen attack.

In code → src/nids/detector/novelty.py : novelty_scores() / choose_threshold()

```
def novelty_scores(detector, X):
    xt = detector.model.transform(X[detector.features])
    return detector.model.kmeans.transform(xt).min(axis=1)   # distance to nearest centre

def choose_threshold(benign_scores, keep_benign=0.95):
    return float(np.quantile(benign_scores, keep_benign))    # tau on benign validation only
```

The code, step by step:

- `kmeans.transform(xt)` returns each flow's distance to **every** centre; `.min(axis=1)` keeps the nearest — the novelty score.
- `np.quantile(benign_scores, 0.95)` is the value below which 95% of benign scores fall, fixing the benign false-novel rate at ~5%.

Worked example. A flow's distances to three centres are [3.2, 9.8, 7.1], so n(x) = 3.2. If τ = 9.5, then 3.2 < 9.5 — **not novel**. Another flow at [11.4, 14.0, 12.1] has n(x) = 11.4 > 9.5 — **novel**.

8. Independent Snort verification

Addresses → Objectives O1, O4 · Research question **RQ2** — independent verification.

Method. The ML verdict is stored **before** Snort is consulted, so Snort never changes it. For each flow we check whether any Snort rule fired, then combine the two independent opinions into Confirmed (both), Disputed (ML normal, Snort fired) or AI-only (ML attack, Snort silent).

In code → scripts/live_detectors/experiment.py : snort_fired() · src/nids/counselor/verification.py

```
snort_fired = df['snort_rules'].astype(str).ne('()')      # did a rule fire on this flow?

def verification_outcome(ml_attack, snort_fired):
    if ml_attack:  return 'confirmed' if snort_fired else 'ai_only'
    return 'disputed' if snort_fired else 'none'
```

The code, step by step:

- `snort_rules` stores the rules that matched; `'()'` means none fired, so `ne('()')` is the Snort verdict.
- The function combines the two verdicts with no feedback between them — this is what makes the verification **independent**.

Worked example. ML says attack and no rule fired → **AI-only** (a possible unseen attack). ML says normal but a rule fired → **Disputed** (Snort caught what the model missed).

9. Alert prioritisation (High / Low)

Addresses → Objective O4 · Research question **RQ2** — reducing false positives.

Method. Confirmed, Disputed, an AI-only that is novel or confident (probability ≥ c), and a normal-but-novel "possible unseen attack" are raised as High; weak AI-only and unresolved alerts are logged as Low. The threshold c is set on validation; false positives are measured on High alerts only.

In code → src/nids/counselor/verification.py : priority()

```
if outcome in ('confirmed','disputed'): return 'high'
if outcome == 'ai_only':  return 'high' if (novel or ml_prob >= c) else 'low'
if novel: return 'high'          # ML said normal but the flow is novel
return 'low' if unverified else 'none'
```

The code, step by step:

- Each branch is one row of the design's priority table — the code **is** the policy.
- Measuring false positives on `'high'` only reflects the alerts an analyst actually acts on.

Worked example. An AI-only flow with probability 0.85 and c = 0.8: 0.85 ≥ 0.8 → **High**. The same outcome at 0.55 and not novel → **Low** (logged, not raised to the analyst).

10. Turning counts into answers (metrics)

Addresses → How the numbers for **every RQ** are computed.

Method. From the four confusion-matrix counts: recall = TP ÷ (TP+FN) is the share of attacks caught; the false-alarm rate = FP ÷ (FP+TN) is the share of benign wrongly raised; MCC combines all four into one score that stays honest under class imbalance.

In code → src/nids/evaluation.py : metrics()

```
recall = tp / (tp + fn)                # detection rate
false_alarm_rate = fp / (fp + tn)
mcc = matthews_corrcoef(y_true, y_pred) # balances both classes
```

The code, step by step:

- `tp, fp, fn, tn` are counted by comparing the boolean prediction array with the true labels.
- MCC is the headline because accuracy looks high even for a model that just labels everything benign.

Worked example. With TP = 90, FN = 10, FP = 5, TN = 900: recall = 90/(90+10) = **0.90**; false-alarm = 5/(5+900) = 0.0055 (**5.5 per 1,000**); MCC = (90×900 – 5×10) / √(95×100×905×910) ≈ **0.92**.

11. Testing RQ1 — leave-one-family-out

Addresses → Objective O3 · Research question **RQ1** — unseen attack types.

Method. To test unseen attacks honestly we remove one whole attack family from training and validation, retrain, and test on that family (now unseen) together with benign. Comparing ML-only with the proposed system on that family is the direct answer to RQ1.

In code → scripts/live_detectors/experiment.py : train_detectors() + run()

```
tr2 = tr[~tr.label.isin(withheld)]          # drop the family from training
va2 = va[~va.label.isin(withheld)]          # and from validation
dets = train_detectors(tr2, va2, feats)     # retrain without it
test_unseen = te[te.label.isin(withheld) | (te.label=='Benign')]   # test on it + benign
```

The code, step by step:

- `~...isin(withheld)` keeps every flow whose label is *not* the withheld family, so the model truly never sees it.
- The test set here is only the withheld family plus benign, so the recall reported is exactly the recall on the unseen family.

Worked example. Withholding DDoS, ML-only caught **27%** of the unseen DDoS flows while the full system caught **88%** — the gap is the contribution of the independent Snort path and the novelty signal.

12. Testing RQ3 — novelty AUROC

Addresses → Objective O3 · Research question **RQ3** — novelty identifies unseen attacks.

Method. AUROC is the probability that a randomly chosen unseen-attack flow has a higher novelty score than a randomly chosen benign flow. 0.5 is a coin flip; higher is better.

In code → src/nids/detector/novelty.py : novelty_auroc()

```
return float(roc_auc_score(is_unseen_attack, scores))   # unseen(1) vs benign(0) by novelty
```

The code, step by step:

- `is_unseen_attack` is the true label (1 unseen attack, 0 benign); `scores` is the novelty score.
- AUROC is threshold-free, so it judges the novelty *signal* itself, separately from τ.

> **Worked example.** If you draw one unseen-attack flow and one benign flow at random and the attack is more novel 82 times out of 100, the AUROC is 0.82 (our runs gave 0.69–0.93).

13. Testing RQ4 — safeguarded adaptation

Addresses → Objective O5 · Research question **RQ4** — safeguarded adaptation (improve a new network without forgetting).

Method. To adapt to a new network without any labels, the system pseudo-labels only the flows both paths **confidently agree on and that are not novel** (attack: Snort and the AI agree; normal: Snort silent, every detector sure, and the flow not novel), then retrains behind a **forgetting gate**. A retrain is **accepted only if nothing gets worse** on the source data — no known attack type loses detection and false alarms do not rise. We train on one dataset, adapt to the other, and compare F1 on the new network before vs after.

In code → scripts/live_detectors/adapt.py (adaptation loop + gate) · src/nids/detector/model.py : refit()

```
labels = agreement_labels(chunk, novelty)      # attack: both agree; normal: agree & NOT novel
candidate = detector.refit(labelled, pseudo_y) # retrain on the agreed flows of the new network
if forgetting(old=detector, new=candidate, source_test, args) == []:
    detector = candidate                        # accept only if no attack type degraded
```

The code, step by step:

- `agreement_labels` leaves novel flows **unlabelled**, so an unfamiliar attack is never taught to the model as “normal” — the blind-spot safeguard.
- `forgetting(...)` returns the attack types whose detection dropped too far on the source test; an empty list means the retrain is safe, so it is accepted, otherwise it is rejected and the current model is kept.

> **Worked example.** Adapting 2017→2018, a proposed retrain would drop DoS-Hulk detection 94%→79% on the source test (beyond the 2% margin), so the gate **rejects** it; a later retrain that lifts live_access 94%→99% with no drop elsewhere is **accepted**.

14. Testing RQ5 — explanation faithfulness

Addresses → Objective O6 · Research question **RQ5** — explanation faithfulness.

Method. We rank a flow's features by their SHAP contribution, neutralise the top-k (set them to their mean), recompute the attack probability and record the drop; we do the same for k random features. If the explanation is faithful, removing the top-k lowers the probability more than removing random features — tested with a Wilcoxon test.

In code → src/nids/detector/faithfulness.py : faithfulness()

```
sv = shap.TreeExplainer(clf).shap_values(x)   # each feature's contribution for each flow
ranked = np.argsort(-np.abs(vals), axis=1)    # rank features by |contribution|
x_top[0, ranked[i,:k]] = 0.0                  # neutralise the top-k
drop = p0[i] - clf.predict_proba(x_top)[0,1]  # fall in attack probability
```

The code, step by step:

- `shap_values` gives, per flow and feature, how much that feature pushed the probability toward attack.
- Setting the chosen features to `0` (their standardised mean) removes their influence without retraining; the drop measures how much they mattered.

Worked example. A flow scores 0.95. Setting its top-5 SHAP features to their mean drops it to 0.39 (a fall of **0.56**); setting 5 random features to their mean drops it only to 0.78 (a fall of **0.17**). The far larger top-5 fall is the evidence the explanation is faithful.